

Project 08: Ride-Sharing Backend

Difficulty: Advanced | Estimated Time: 3 Weeks

1. Project Overview and Goals

This project builds the database backend for a ride-sharing platform comparable to Uber or Lyft. It involves geospatial data for driver/rider location tracking, real-time matching logic, dynamic pricing (surge), trip lifecycle management, driver earnings, safety features, and comprehensive analytics. This is one of the most technically demanding projects due to its geospatial and real-time requirements.

Goals:

- Use PostGIS for geospatial queries (nearby drivers, trip distance, service zones).
 - Model a real-time driver location system with time-decay queries.
 - Implement surge pricing based on supply/demand ratio in geographic cells.
 - Design a trip lifecycle state machine with full audit history.
 - Build driver earnings and settlement calculations.
 - Write dispatcher matching algorithms in SQL.
-

2. Learning Objectives

- Install and use the PostGIS extension for geographic data types.
 - Use `GEOMETRY(POINT)` and `ST_DWithin`, `ST_Distance` for proximity queries.
 - Understand geohashing with `ST_GeoHash` for spatial indexing.
 - Implement time-windowed aggregations for surge pricing.
 - Use `LISTEN/NOTIFY` pattern via triggers for real-time driver updates.
 - Write complex CASE-based dynamic pricing formulas.
 - Practice geo-fence zone queries with polygon types.
 - Implement driver rating systems with weighted averages.
-

3. Functional Requirements

- **Drivers:** Registration, vehicle details, document verification, real-time location.
 - **Riders:** Profile, payment methods, home/work addresses.
 - **Trips:** Request, match, pickup, dropoff lifecycle.
 - **Pricing:** Base fare + per-mile + per-minute + surge multiplier.
 - **Payments:** Trip payments, driver payouts, platform commission.
 - **Ratings:** Mutual rating system (driver rates rider, rider rates driver).
 - **Zones:** Service area polygon definition and geofencing.
 - **Safety:** Trip sharing, emergency contact, incident reports.
 - **Analytics:** ETA accuracy, revenue per zone, driver utilization.
-

4. Non-Functional Requirements

- Driver locations updated every 10 seconds — only latest position stored.
- Geospatial queries use GIST indexes on geometry columns.
- Trip prices stored in cents (`INTEGER`).
- Driver matching query must return results in under 100ms.

- All location data stored as WGS84 (SRID 4326).
- Time zones stored per city in a zones table.

5. Complete Database Schema

```
-- =====
-- RIDE-SHARING BACKEND - Complete Schema
-- PostgreSQL 15+ with PostGIS
-- =====

CREATE SCHEMA IF NOT EXISTS rides;
SET search_path = rides, public;

CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS postgis_topology;

-----
-- VEHICLE CATEGORIES
-----

CREATE TABLE vehicle_categories (
    category_id    SERIAL PRIMARY KEY,
    name           VARCHAR(50) NOT NULL UNIQUE,
    description     TEXT,
    base_fare_cents INTEGER NOT NULL DEFAULT 200,
    per_mile_cents  INTEGER NOT NULL DEFAULT 100,
    per_min_cents   INTEGER NOT NULL DEFAULT 25,
    min_fare_cents  INTEGER NOT NULL DEFAULT 500,
    cancellation_fee_cents INTEGER NOT NULL DEFAULT 300,
    capacity        SMALLINT NOT NULL DEFAULT 4
);

INSERT INTO vehicle_categories (name, description, base_fare_cents, per_mile_cents,
per_min_cents, capacity)
VALUES
    ('Economy',    'Affordable everyday rides',    200, 80, 18, 4),
    ('Comfort',    'Newer cars with extra legroom', 300, 120, 25, 4),
    ('Premium',    'Luxury vehicles',              500, 200, 40, 4),
    ('XL',         'SUVs for groups up to 6',      350, 130, 28, 6),
    ('Pool',       'Share your ride, save money',  150, 60, 15, 4),
    ('Moto',       'Two-wheel rides for one',      100, 50, 12, 1);

-----
-- SERVICE ZONES (geofenced areas)
-----

CREATE TABLE service_zones (
    zone_id        SERIAL PRIMARY KEY,
    name           VARCHAR(200) NOT NULL,
    city           VARCHAR(100) NOT NULL,
    country        VARCHAR(100) NOT NULL DEFAULT 'USA',
    timezone       VARCHAR(50) NOT NULL DEFAULT 'America/New_York',
```

```

    boundary      GEOMETRY(POLYGON, 4326) NOT NULL,
    is_active      BOOLEAN NOT NULL DEFAULT TRUE,
    surge_enabled  BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE INDEX idx_zones_boundary ON service_zones USING GIST (boundary);

-----
-- DRIVERS
-----

CREATE TABLE drivers (
    driver_id      SERIAL PRIMARY KEY,
    driver_uuid    UUID NOT NULL DEFAULT gen_random_uuid() UNIQUE,
    first_name     VARCHAR(100) NOT NULL,
    last_name      VARCHAR(100) NOT NULL,
    email          VARCHAR(300) NOT NULL UNIQUE,
    phone          VARCHAR(30) NOT NULL UNIQUE,
    date_of_birth  DATE NOT NULL,
    license_number VARCHAR(50) NOT NULL UNIQUE,
    license_expiry DATE NOT NULL,
    status         VARCHAR(20) NOT NULL DEFAULT 'Pending'
                CHECK (status IN
('Pending', 'Active', 'Suspended', 'Deactivated')),
    rating         NUMERIC(3,2) NOT NULL DEFAULT 5.00 CHECK (rating BETWEEN 1.00
AND 5.00),
    total_trips    INTEGER NOT NULL DEFAULT 0,
    total_earnings_cents BIGINT NOT NULL DEFAULT 0,
    acceptance_rate NUMERIC(5,2) NOT NULL DEFAULT 100.00,
    cancellation_rate NUMERIC(5,2) NOT NULL DEFAULT 0.00,
    home_location  GEOMETRY(POINT, 4326),
    created_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_drivers_status ON drivers (status) WHERE status = 'Active';
CREATE INDEX idx_drivers_rating ON drivers (rating DESC);

-----
-- VEHICLES
-----

CREATE TABLE vehicles (
    vehicle_id     SERIAL PRIMARY KEY,
    driver_id      INTEGER NOT NULL REFERENCES drivers(driver_id),
    category_id    INTEGER NOT NULL REFERENCES vehicle_categories(category_id),
    make           VARCHAR(50) NOT NULL,
    model          VARCHAR(50) NOT NULL,
    year           SMALLINT NOT NULL CHECK (year BETWEEN 2010 AND 2030),
    color          VARCHAR(30) NOT NULL,
    license_plate  VARCHAR(20) NOT NULL UNIQUE,
    vin            VARCHAR(17),
    is_primary     BOOLEAN NOT NULL DEFAULT TRUE,
    inspection_expiry DATE,
    is_active      BOOLEAN NOT NULL DEFAULT TRUE

```

```

);

CREATE INDEX idx_vehicles_driver ON vehicles (driver_id) WHERE is_active = TRUE;

-----
-- DRIVER LOCATIONS (real-time, only latest per driver)
-----

CREATE TABLE driver_locations (
    driver_id    INTEGER PRIMARY KEY REFERENCES drivers(driver_id),
    location     GEOMETRY(POINT, 4326) NOT NULL,
    heading      NUMERIC(5,2),    -- degrees 0-360
    speed_mph    NUMERIC(5,1),
    is_online    BOOLEAN NOT NULL DEFAULT FALSE,
    is_available BOOLEAN NOT NULL DEFAULT FALSE,
    zone_id      INTEGER REFERENCES service_zones(zone_id),
    updated_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_driver_locations_geom ON driver_locations USING GIST (location);
CREATE INDEX idx_driver_available ON driver_locations (is_available)
    WHERE is_available = TRUE;

-----
-- RIDERS
-----

CREATE TABLE riders (
    rider_id     SERIAL PRIMARY KEY,
    rider_uuid   UUID NOT NULL DEFAULT gen_random_uuid() UNIQUE,
    first_name   VARCHAR(100) NOT NULL,
    last_name    VARCHAR(100) NOT NULL,
    email        VARCHAR(300) NOT NULL UNIQUE,
    phone        VARCHAR(30) NOT NULL UNIQUE,
    rating       NUMERIC(3,2) NOT NULL DEFAULT 5.00 CHECK (rating BETWEEN 1.00 AND
5.00),
    total_trips  INTEGER NOT NULL DEFAULT 0,
    home_address TEXT,
    home_location GEOMETRY(POINT, 4326),
    work_location GEOMETRY(POINT, 4326),
    is_active    BOOLEAN NOT NULL DEFAULT TRUE,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-----
-- SURGE PRICING HISTORY
-----

CREATE TABLE surge_events (
    surge_id     BIGSERIAL PRIMARY KEY,
    zone_id      INTEGER NOT NULL REFERENCES service_zones(zone_id),
    multiplier    NUMERIC(3,2) NOT NULL DEFAULT 1.00 CHECK (multiplier BETWEEN 1.00
AND 10.00),
    demand_count INTEGER NOT NULL,    -- active requests in zone
    supply_count  INTEGER NOT NULL,   -- available drivers in zone

```

```

        started_at    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
        ended_at      TIMESTAMPTZ
    );

CREATE INDEX idx_surge_zone_time ON surge_events (zone_id, started_at DESC);

-----
-- TRIPS
-----

CREATE TABLE trips (
    trip_id           BIGSERIAL PRIMARY KEY,
    trip_uuid         UUID NOT NULL DEFAULT gen_random_uuid() UNIQUE,
    rider_id          INTEGER NOT NULL REFERENCES riders(rider_id),
    driver_id         INTEGER REFERENCES drivers(driver_id),
    vehicle_id        INTEGER REFERENCES vehicles(vehicle_id),
    category_id       INTEGER NOT NULL REFERENCES
vehicle_categories(vehicle_category_id),
    zone_id           INTEGER REFERENCES service_zones(zone_id),
    status            VARCHAR(20) NOT NULL DEFAULT 'Requested'
                    CHECK (status IN
('Requested', 'Matched', 'Arriving', 'InProgress',
                    'Completed', 'Cancelled', 'NoDriverFound')),
    pickup_address    VARCHAR(500) NOT NULL,
    pickup_location    GEOMETRY(POINT, 4326) NOT NULL,
    dropoff_address    VARCHAR(500) NOT NULL,
    dropoff_location   GEOMETRY(POINT, 4326) NOT NULL,
    actual_path        GEOMETRY(LINESTRING, 4326),
    distance_miles     NUMERIC(8,3),
    duration_minutes   NUMERIC(8,2),
    -- Pricing
    base_fare_cents    INTEGER NOT NULL DEFAULT 0,
    distance_fare_cents INTEGER NOT NULL DEFAULT 0,
    time_fare_cents    INTEGER NOT NULL DEFAULT 0,
    surge_multiplier   NUMERIC(3,2) NOT NULL DEFAULT 1.00,
    surge_fare_cents   INTEGER NOT NULL DEFAULT 0,
    promo_discount_cents INTEGER NOT NULL DEFAULT 0,
    total_fare_cents   INTEGER NOT NULL DEFAULT 0,
    driver_earnings_cents INTEGER NOT NULL DEFAULT 0,
    platform_fee_cents INTEGER NOT NULL DEFAULT 0,
    -- Timestamps
    requested_at       TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    matched_at         TIMESTAMPTZ,
    driver_arrived_at  TIMESTAMPTZ,
    started_at         TIMESTAMPTZ,
    completed_at       TIMESTAMPTZ,
    cancelled_at       TIMESTAMPTZ,
    cancel_reason      VARCHAR(200),
    -- ETA
    estimated_pickup_min SMALLINT,
    estimated_duration_min SMALLINT,
    -- Ratings
    rider_rating       SMALLINT CHECK (rider_rating BETWEEN 1 AND 5),

```

```

        driver_rating    SMALLINT CHECK (driver_rating BETWEEN 1 AND 5),
        rider_tip_cents  INTEGER NOT NULL DEFAULT 0
    );

COMMENT ON COLUMN trips.category_id IS 'Vehicle category column alias for FK';
ALTER TABLE trips RENAME COLUMN category_id TO vehicle_category_id;

CREATE INDEX idx_trips_rider    ON trips (rider_id, requested_at DESC);
CREATE INDEX idx_trips_driver  ON trips (driver_id, requested_at DESC);
CREATE INDEX idx_trips_status  ON trips (status, requested_at DESC);
CREATE INDEX idx_trips_pickup  ON trips USING GIST (pickup_location);
CREATE INDEX idx_trips_dropoff ON trips USING GIST (dropoff_location);
CREATE INDEX idx_trips_zone    ON trips (zone_id, requested_at DESC);

-- -----
-- TRIP STATUS HISTORY
-- -----
CREATE TABLE trip_status_history (
    history_id  BIGSERIAL PRIMARY KEY,
    trip_id     BIGINT NOT NULL REFERENCES trips(trip_id),
    status      VARCHAR(20) NOT NULL,
    location    GEOMETRY(POINT, 4326),
    recorded_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_trip_history ON trip_status_history (trip_id, recorded_at);

-- -----
-- DRIVER PAYOUTS
-- -----
CREATE TABLE driver_payouts (
    payout_id    BIGSERIAL PRIMARY KEY,
    driver_id    INTEGER NOT NULL REFERENCES drivers(driver_id),
    period_start DATE NOT NULL,
    period_end   DATE NOT NULL,
    total_trips  INTEGER NOT NULL,
    gross_earnings_cents BIGINT NOT NULL,
    platform_fees_cents BIGINT NOT NULL,
    tips_cents   BIGINT NOT NULL,
    net_payout_cents BIGINT NOT NULL,
    status       VARCHAR(20) NOT NULL DEFAULT 'Pending'
                CHECK (status IN ('Pending', 'Processing', 'Paid', 'Failed')),
    initiated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    paid_at      TIMESTAMPTZ
);

-- -----
-- INCIDENT REPORTS
-- -----
CREATE TABLE incidents (
    incident_id  SERIAL PRIMARY KEY,
    trip_id      BIGINT NOT NULL REFERENCES trips(trip_id),

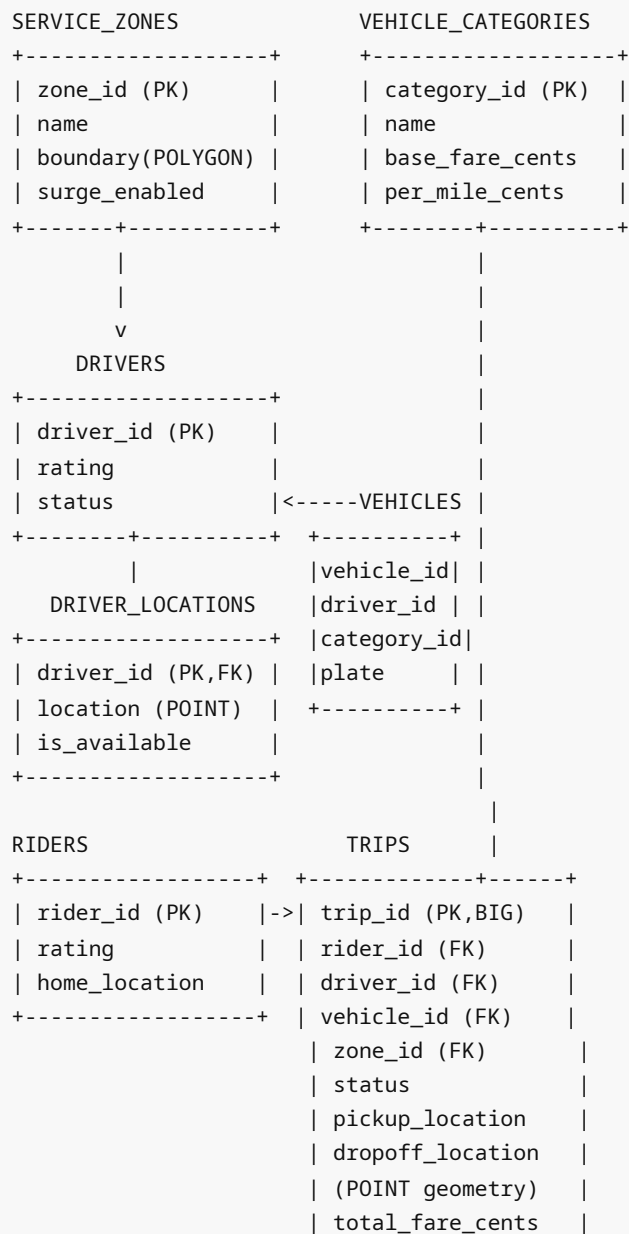
```

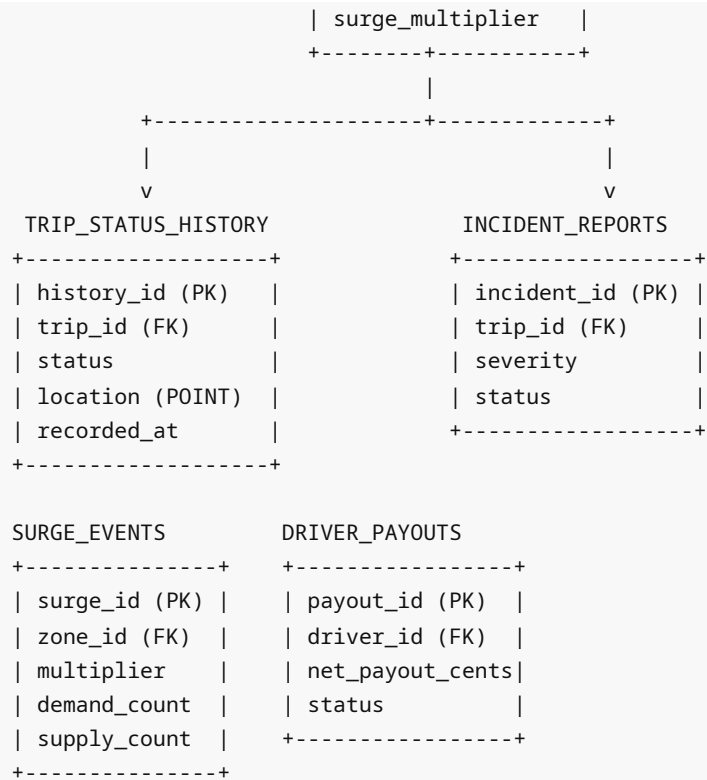
```

reporter_type VARCHAR(10) NOT NULL CHECK (reporter_type IN ('driver','rider')),
reporter_id   INTEGER NOT NULL,
type         VARCHAR(50) NOT NULL,
description   TEXT,
severity      VARCHAR(10) CHECK (severity IN
('Low','Medium','High','Critical')),
location      GEOMETRY(POINT, 4326),
status        VARCHAR(20) NOT NULL DEFAULT 'Open'
CHECK (status IN ('Open','Investigating','Resolved','Closed')),
created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

6. ASCII ER Diagram





7. Sample Data Scripts

```

SET search_path = rides, public;

-- Service zone (approximate NYC boundary polygon)
INSERT INTO service_zones (name, city, timezone, boundary) VALUES
('NYC Metro', 'New York', 'America/New_York',
ST_GeomFromText('POLYGON((-74.25 40.49,-73.70 40.49,-73.70 40.92,-74.25
40.92,-74.25 40.49))', 4326)),
('Chicago', 'Chicago', 'America/Chicago',
ST_GeomFromText('POLYGON((-87.94 41.64,-87.52 41.64,-87.52 42.02,-87.94
42.02,-87.94 41.64))', 4326));

-- Drivers
INSERT INTO drivers (first_name, last_name, email, phone, date_of_birth,
license_number, license_expiry, status) VALUES
('Carlos', 'Rivera', 'carlos.r@drivers.com', '917-555-1001', '1985-04-12', 'DL-
001-NY', '2026-04-12', 'Active'),
('Priya', 'Singh', 'priya.s@drivers.com', '917-555-1002', '1990-08-22', 'DL-
002-NY', '2027-08-22', 'Active'),
('James', 'Walker', 'james.w@drivers.com', '312-555-2001', '1980-11-05', 'DL-
003-IL', '2025-11-05', 'Active'),
('Lin', 'Zhang', 'lin.z@drivers.com', '917-555-1003', '1992-03-17', 'DL-
004-NY', '2026-03-17', 'Active');

-- Vehicles

```

```

INSERT INTO vehicles (driver_id, category_id, make, model, year, color,
license_plate) VALUES
  (1, 1, 'Toyota', 'Camry', 2021, 'Silver', 'NYC-1234'),
  (2, 2, 'Honda', 'Accord', 2022, 'Black', 'NYC-5678'),
  (3, 4, 'Chevrolet', 'Suburban', 2020, 'White', 'ILL-9012'),
  (4, 1, 'Hyundai', 'Elantra', 2023, 'Blue', 'NYC-3456');

-- Driver locations (current positions in NYC)
INSERT INTO driver_locations (driver_id, location, heading, speed_mph, is_online,
is_available, zone_id) VALUES
  (1, ST_SetSRID(ST_MakePoint(-73.9857, 40.7580), 4326), 90, 15.5, TRUE, TRUE, 1),
-- Midtown
  (2, ST_SetSRID(ST_MakePoint(-73.9442, 40.6782), 4326), 180, 0.0, TRUE, TRUE, 1),
-- Brooklyn
  (3, ST_SetSRID(ST_MakePoint(-87.6298, 41.8781), 4326), 270, 20.0, TRUE, TRUE, 2),
-- Chicago Loop
  (4, ST_SetSRID(ST_MakePoint(-73.9857, 40.7484), 4326), 0, 5.0, TRUE, FALSE, 1);
-- On a trip

-- Riders
INSERT INTO riders (first_name, last_name, email, phone) VALUES
  ('Alice', 'Johnson', 'alice@riders.com', '646-555-0101'),
  ('Marcus', 'Brown', 'marcus@riders.com', '646-555-0102'),
  ('Sophie', 'Lee', 'sophie@riders.com', '312-555-0201');

-- Trips (historical)
INSERT INTO trips (rider_id, driver_id, vehicle_id, vehicle_category_id, zone_id,
status,
pickup_address, pickup_location, dropoff_address,
dropoff_location,
distance_miles, duration_minutes, base_fare_cents,
distance_fare_cents,
time_fare_cents, total_fare_cents, driver_earnings_cents,
platform_fee_cents,
surge_multiplier, requested_at, started_at, completed_at,
rider_rating, driver_rating)
VALUES
  (1, 1, 1, 1, 1, 'Completed',
  'Times Square, NYC', ST_SetSRID(ST_MakePoint(-73.9857, 40.7580), 4326),
  'Central Park South, NYC', ST_SetSRID(ST_MakePoint(-73.9765, 40.7656), 4326),
  1.2, 8.5, 200, 96, 153, 449, 336, 113, 1.00,
  NOW()-INTERVAL '2 hours', NOW()-INTERVAL '115 min', NOW()-INTERVAL '1 hour 47
min',
  5, 5),
  (2, 2, 2, 2, 1, 'Completed',
  'JFK Airport, NYC', ST_SetSRID(ST_MakePoint(-73.7781, 40.6413), 4326),
  'Midtown Manhattan', ST_SetSRID(ST_MakePoint(-73.9857, 40.7580), 4326),
  15.8, 42.0, 300, 1896, 1050, 3900, 2925, 975, 1.25,
  NOW()-INTERVAL '5 hours', NOW()-INTERVAL '4 hours 45 min', NOW()-INTERVAL '4
hours',
  4, 5);

```

8. Core SQL Queries

```
SET search_path = rides, public;

-- -----
-- Q1: Find nearest available drivers to a pickup point
-- -----

WITH pickup AS (
    SELECT ST_SetSRID(ST_MakePoint(-73.9857, 40.7580), 4326) AS point
)
SELECT
    d.driver_id,
    d.first_name || ' ' || d.last_name AS driver,
    d.rating,
    vc.name AS vehicle_category,
    v.make || ' ' || v.model || ' (' || v.color || ')' AS vehicle,
    ROUND(ST_Distance(dl.location::GEOGRAPHY, p.point::GEOGRAPHY) / 1609.34, 2) AS
distance_miles,
    ROUND(ST_Distance(dl.location::GEOGRAPHY, p.point::GEOGRAPHY) / 1609.34 / 15 *
60, 0) AS eta_minutes
FROM driver_locations dl
JOIN drivers d ON dl.driver_id = d.driver_id
JOIN vehicles v ON d.driver_id = v.driver_id AND v.is_primary = TRUE
JOIN vehicle_categories vc ON v.category_id = vc.category_id,
pickup p
WHERE dl.is_online = TRUE
    AND dl.is_available = TRUE
    AND d.status = 'Active'
    AND ST_DWithin(dl.location::GEOGRAPHY, p.point::GEOGRAPHY, 8046.72) -- 5 miles in
meters
ORDER BY dl.location <-> p.point -- KNN operator for fast geo sort
LIMIT 5;

-- -----
-- Q2: Calculate trip fare before booking
-- -----

WITH route AS (
    SELECT
        ST_SetSRID(ST_MakePoint(-73.9857, 40.7580), 4326) AS pickup,
        ST_SetSRID(ST_MakePoint(-73.9442, 40.6782), 4326) AS dropoff
),
distance AS (
    SELECT
        ROUND(ST_Distance(r.pickup::GEOGRAPHY, r.dropoff::GEOGRAPHY) / 1609.34, 2)
AS miles
    FROM route r
)
SELECT
    vc.name AS category,
    d.miles AS distance,
```

```

        vc.base_fare_cents / 100.0                AS base_fare,
        ROUND(d.miles * vc.per_mile_cents / 100.0, 2) AS distance_fare,
        ROUND(d.miles / 25 * 60 * vc.per_min_cents / 100.0, 2) AS time_fare_estimate,
        ROUND(GREATEST(
            vc.min_fare_cents,
            vc.base_fare_cents + d.miles * vc.per_mile_cents + (d.miles / 25 * 60) *
vc.per_min_cents
        ) / 100.0, 2) AS estimated_fare
FROM vehicle_categories vc, distance d
ORDER BY vc.base_fare_cents;

-----
-- Q3: Current surge multiplier for a zone
-----

WITH zone_supply AS (
    SELECT dl.zone_id, COUNT(*) AS available_drivers
    FROM driver_locations dl
    WHERE dl.is_available = TRUE
    GROUP BY dl.zone_id
),
zone_demand AS (
    SELECT t.zone_id, COUNT(*) AS active_requests
    FROM trips t
    WHERE t.status = 'Requested'
    GROUP BY t.zone_id
)
SELECT
    sz.name AS zone,
    COALESCE(d.active_requests, 0) AS demand,
    COALESCE(s.available_drivers, 0) AS supply,
    ROUND(CASE
        WHEN COALESCE(s.available_drivers, 0) = 0 THEN 3.00
        WHEN COALESCE(d.active_requests, 0)::FLOAT / COALESCE(s.available_drivers,
1) > 3 THEN 2.50
        WHEN COALESCE(d.active_requests, 0)::FLOAT / COALESCE(s.available_drivers,
1) > 2 THEN 1.75
        WHEN COALESCE(d.active_requests, 0)::FLOAT / COALESCE(s.available_drivers,
1) > 1 THEN 1.25
        ELSE 1.00
    END, 2) AS current_surge
FROM service_zones sz
LEFT JOIN zone_supply s ON sz.zone_id = s.zone_id
LEFT JOIN zone_demand d ON sz.zone_id = d.zone_id
WHERE sz.is_active = TRUE
ORDER BY current_surge DESC;

-----
-- Q4: Driver daily earnings summary
-----

SELECT
    d.first_name || ' ' || d.last_name AS driver,
    DATE(t.completed_at) AS date,

```

```

COUNT(*) AS trips,
ROUND(SUM(t.driver_earnings_cents) / 100.0, 2) AS earnings,
ROUND(SUM(t.rider_tip_cents) / 100.0, 2) AS tips,
ROUND(SUM(t.distance_miles), 1) AS total_miles,
ROUND(AVG(t.duration_minutes), 1) AS avg_trip_min,
ROUND(AVG(t.driver_rating), 2) AS avg_rating
FROM trips t
JOIN drivers d ON t.driver_id = d.driver_id
WHERE t.status = 'Completed'
AND t.completed_at >= CURRENT_DATE - 7
GROUP BY d.driver_id, d.first_name, d.last_name, DATE(t.completed_at)
ORDER BY date DESC, earnings DESC;

-- -----
-- Q5: ETA accuracy analysis
-- -----
SELECT
    t.estimated_pickup_min AS eta_promised,
    ROUND(EXTRACT(EPOCH FROM (t.driver_arrived_at - t.matched_at)) / 60, 1) AS
actual_minutes,
    ROUND(EXTRACT(EPOCH FROM (t.driver_arrived_at - t.matched_at)) / 60
        - t.estimated_pickup_min, 1) AS eta_error_min,
    CASE
        WHEN ABS(EXTRACT(EPOCH FROM (t.driver_arrived_at - t.matched_at)) / 60
            - t.estimated_pickup_min) <= 2 THEN 'Accurate'
        WHEN EXTRACT(EPOCH FROM (t.driver_arrived_at - t.matched_at)) / 60
            > t.estimated_pickup_min THEN 'Late'
        ELSE 'Early'
    END AS accuracy_status
FROM trips t
WHERE t.driver_arrived_at IS NOT NULL
AND t.estimated_pickup_min IS NOT NULL
AND t.status = 'Completed';

-- -----
-- Q6: Heatmap data - trip density by area
-- -----
SELECT
    ROUND(ST_X(pickup_location)::NUMERIC, 2) AS lon_bin,
    ROUND(ST_Y(pickup_location)::NUMERIC, 2) AS lat_bin,
    COUNT(*) AS trip_count,
    ROUND(AVG(total_fare_cents) / 100.0, 2) AS avg_fare
FROM trips
WHERE status = 'Completed'
AND requested_at >= NOW() - INTERVAL '7 days'
GROUP BY lon_bin, lat_bin
ORDER BY trip_count DESC;

-- -----
-- Q7: Driver utilization rate
-- -----
SELECT

```

```

    d.first_name || ' ' || d.last_name      AS driver,
    COUNT(t.trip_id)                        AS completed_trips,
    ROUND(SUM(t.duration_minutes), 0)       AS total_trip_minutes,
    ROUND(SUM(t.duration_minutes) / (8 * 60) * 100, 1) AS utilization_pct,
    -- Assume 8-hour shift
    ROUND(SUM(t.total_fare_cents) / 100.0, 2) AS gross_revenue,
    ROUND(SUM(t.driver_earnings_cents) / 100.0, 2) AS driver_take
FROM drivers d
LEFT JOIN trips t ON d.driver_id = t.driver_id
    AND t.status = 'Completed'
    AND t.completed_at >= CURRENT_DATE
GROUP BY d.driver_id, d.first_name, d.last_name
ORDER BY utilization_pct DESC NULLS LAST;

```

-- Q8: Revenue by zone and hour of day

```

SELECT
    sz.name                                AS zone,
    EXTRACT(HOUR FROM t.requested_at)      AS hour_of_day,
    COUNT(*)                              AS trips,
    ROUND(SUM(t.total_fare_cents) / 100.0, 2) AS revenue,
    ROUND(AVG(t.surge_multiplier), 2)     AS avg_surge
FROM trips t
JOIN service_zones sz ON t.zone_id = sz.zone_id
WHERE t.status = 'Completed'
GROUP BY sz.zone_id, sz.name, EXTRACT(HOUR FROM t.requested_at)
ORDER BY sz.name, hour_of_day;

```

-- Q9: Cancellation rate analysis

```

SELECT
    d.first_name || ' ' || d.last_name      AS driver,
    COUNT(*)                                AS total_requests,
    COUNT(*) FILTER (WHERE t.status = 'Completed') AS completed,
    COUNT(*) FILTER (WHERE t.status = 'Cancelled') AS cancelled,
    ROUND(100.0 * COUNT(*) FILTER (WHERE t.status = 'Cancelled')
          / NULLIF(COUNT(*), 0), 1)         AS cancel_rate_pct
FROM drivers d
LEFT JOIN trips t ON d.driver_id = t.driver_id
    AND t.driver_id IS NOT NULL
WHERE d.status = 'Active'
GROUP BY d.driver_id, d.first_name, d.last_name
ORDER BY cancel_rate_pct DESC NULLS LAST;

```

-- Q10: Trips that cross zone boundaries (geospatial)

```

SELECT
    t.trip_id,
    sz_pickup.name AS pickup_zone,

```

```

        sz_dropoff.name AS dropoff_zone,
        t.distance_miles,
        t.total_fare_cents / 100.0 AS fare
FROM trips t
JOIN service_zones sz_pickup ON ST_Within(t.pickup_location, sz_pickup.boundary)
JOIN service_zones sz_dropoff ON ST_Within(t.dropoff_location, sz_dropoff.boundary)
WHERE sz_pickup.zone_id <> sz_dropoff.zone_id
      AND t.status = 'Completed';

-- -----
-- Q11: Driver performance tiers (window function ranking)
-- -----

SELECT
    d.first_name || ' ' || d.last_name AS driver,
    d.rating,
    d.total_trips,
    d.acceptance_rate,
    NTILE(4) OVER (ORDER BY
        d.rating * 0.4 + d.acceptance_rate / 100 * 0.3
        + LEAST(d.total_trips, 1000) / 1000.0 * 0.3
    DESC) AS performance_tier,
    -- Tier 1=top, 4=bottom
    ROUND(d.rating * 0.4 + d.acceptance_rate / 100 * 0.3
        + LEAST(d.total_trips, 1000) / 1000.0 * 0.3, 3) AS composite_score
FROM drivers d
WHERE d.status = 'Active'
ORDER BY composite_score DESC;

-- -----
-- Q12: Popular routes (most common pickup-dropoff pairs)
-- -----

SELECT
    pickup_address,
    dropoff_address,
    COUNT(*) AS trip_count,
    ROUND(AVG(total_fare_cents) / 100.0, 2) AS avg_fare,
    ROUND(AVG(duration_minutes), 1) AS avg_duration_min
FROM trips
WHERE status = 'Completed'
GROUP BY pickup_address, dropoff_address
HAVING COUNT(*) > 1
ORDER BY trip_count DESC
LIMIT 20;

```

9. Stored Procedures / Functions

```

-- -----
-- FUNCTION 1: Match a rider to nearest available driver
-- -----

CREATE OR REPLACE FUNCTION rides.match_trip(

```

```

    p_rider_id          INTEGER,
    p_pickup_location    GEOMETRY(POINT, 4326),
    p_category_id        INTEGER,
    p_max_wait_miles     FLOAT DEFAULT 5.0
)
RETURNS TABLE (
    driver_id           INTEGER,
    driver_name         TEXT,
    vehicle_info        TEXT,
    eta_minutes         INTEGER,
    distance_miles      FLOAT
)
LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
    SELECT
        d.driver_id,
        d.first_name || ' ' || d.last_name,
        v.make || ' ' || v.model || ' ' || v.color || ' (' || v.license_plate ||
    ')',
        ROUND(ST_Distance(dl.location::GEOGRAPHY, p_pickup_location::GEOGRAPHY) /
1609.34 / 15 * 60)::INTEGER,
        ROUND(ST_Distance(dl.location::GEOGRAPHY, p_pickup_location::GEOGRAPHY) /
1609.34, 2)::FLOAT
    FROM rides.driver_locations dl
    JOIN rides.drivers d    ON dl.driver_id = d.driver_id
    JOIN rides.vehicles v   ON d.driver_id = v.driver_id AND v.is_primary = TRUE AND
v.category_id = p_category_id
    WHERE dl.is_available = TRUE
        AND dl.is_online   = TRUE
        AND d.status       = 'Active'
        AND ST_DWithin(dl.location::GEOGRAPHY, p_pickup_location::GEOGRAPHY,
p_max_wait_miles * 1609.34)
        AND NOT EXISTS (
            SELECT 1 FROM rides.trips t2
            WHERE t2.driver_id = d.driver_id
                AND t2.status IN ('Matched', 'Arriving', 'InProgress')
        )
    ORDER BY dl.location <-> p_pickup_location
    LIMIT 1;
END;
$$;

-- -----
-- FUNCTION 2: Complete a trip and calculate final fare
-- -----

CREATE OR REPLACE FUNCTION rides.complete_trip(
    p_trip_id          BIGINT,
    p_actual_path       GEOMETRY(LINESTRING, 4326),
    p_duration_min      NUMERIC
)
RETURNS TABLE (total_fare_usd NUMERIC, driver_earnings_usd NUMERIC)

```

```

LANGUAGE plpgsql AS $$
DECLARE
    v_trip      rides.trips%ROWTYPE;
    v_vc        rides.vehicle_categories%ROWTYPE;
    v_distance  NUMERIC(8,3);
    v_base      INTEGER;
    v_dist_fare INTEGER;
    v_time_fare INTEGER;
    v_subtotal  INTEGER;
    v_surge     NUMERIC(3,2);
    v_total     INTEGER;
    v_driver_cut INTEGER;
    v_platform  INTEGER;
BEGIN
    SELECT * INTO v_trip FROM rides.trips WHERE trip_id = p_trip_id FOR UPDATE;
    IF NOT FOUND THEN RAISE EXCEPTION 'Trip % not found', p_trip_id; END IF;
    IF v_trip.status <> 'InProgress' THEN
        RAISE EXCEPTION 'Trip % is not in progress (status: %)', p_trip_id,
v_trip.status;
    END IF;

    SELECT * INTO v_vc FROM rides.vehicle_categories WHERE category_id =
v_trip.vehicle_category_id;

    -- Calculate actual distance from path
    v_distance := ROUND(ST_Length(p_actual_path::GEOGRAPHY) / 1609.34, 3);

    v_base      := v_vc.base_fare_cents;
    v_dist_fare := (v_distance * v_vc.per_mile_cents)::INTEGER;
    v_time_fare := (p_duration_min * v_vc.per_min_cents)::INTEGER;
    v_subtotal  := GREATEST(v_vc.min_fare_cents, v_base + v_dist_fare +
v_time_fare);
    v_surge     := v_trip.surge_multiplier;
    v_total     := (v_subtotal * v_surge)::INTEGER;

    -- Driver gets 75%, platform 25%
    v_driver_cut := (v_total * 0.75)::INTEGER;
    v_platform   := v_total - v_driver_cut;

    UPDATE rides.trips
    SET status          = 'Completed',
        actual_path     = p_actual_path,
        distance_miles  = v_distance,
        duration_minutes = p_duration_min,
        base_fare_cents = v_base,
        distance_fare_cents = v_dist_fare,
        time_fare_cents = v_time_fare,
        total_fare_cents = v_total,
        driver_earnings_cents = v_driver_cut,
        platform_fee_cents = v_platform,
        completed_at    = NOW()
    WHERE trip_id = p_trip_id;

```

```

UPDATE rides.drivers
SET total_trips          = total_trips + 1,
    total_earnings_cents = total_earnings_cents + v_driver_cut
WHERE driver_id = v_trip.driver_id;

UPDATE rides.driver_locations
SET is_available = TRUE
WHERE driver_id = v_trip.driver_id;

RETURN QUERY SELECT v_total / 100.0, v_driver_cut / 100.0;
END;
$$;

```

10. Triggers

```

-- TRIGGER 1: Notify when driver location updated (LISTEN/NOTIFY)
CREATE OR REPLACE FUNCTION rides.trg_driver_location_notify()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    PERFORM pg_notify(
        'driver_location_update',
        json_build_object(
            'driver_id', NEW.driver_id,
            'lat',       ST_Y(NEW.location),
            'lon',       ST_X(NEW.location),
            'available', NEW.is_available,
            'updated_at', NEW.updated_at
        )::TEXT
    );
    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_driver_loc_notify
AFTER UPDATE ON rides.driver_locations
FOR EACH ROW EXECUTE FUNCTION rides.trg_driver_location_notify();

-- TRIGGER 2: Log trip status changes
CREATE OR REPLACE FUNCTION rides.trg_log_trip_status()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
    IF NEW.status IS DISTINCT FROM OLD.status THEN
        INSERT INTO rides.trip_status_history (trip_id, status, recorded_at)
        VALUES (NEW.trip_id, NEW.status, NOW());
    END IF;
    RETURN NEW;
END;
$$;

```

```
CREATE TRIGGER trg_trip_status_history
AFTER UPDATE OF status ON rides.trips
FOR EACH ROW EXECUTE FUNCTION rides.trg_log_trip_status();
```

11. Performance Optimization

```
-- KNN (K-Nearest Neighbor) operator for driver search
-- The <-> operator with GIST index is critical for sub-100ms matching
CREATE INDEX idx_driver_loc_knn ON rides.driver_locations
    USING GIST (location)
    WHERE is_available = TRUE;

-- Clustered index on trips by zone+date for zone analytics
CREATE INDEX idx_trips_zone_date ON rides.trips (zone_id, requested_at DESC)
    WHERE status = 'Completed';

-- Covering index for driver earnings queries
CREATE INDEX idx_trips_driver_earnings ON rides.trips (driver_id, completed_at DESC)
    INCLUDE (driver_earnings_cents, rider_tip_cents, distance_miles,
    duration_minutes)
    WHERE status = 'Completed';
```

12. Extensions Used

```
CREATE EXTENSION IF NOT EXISTS postgis;           -- Core geospatial extension
CREATE EXTENSION IF NOT EXISTS postgis_topology;  -- Advanced topology operations
CREATE EXTENSION IF NOT EXISTS pgcrypto;          -- gen_random_uuid() for UUIDs
CREATE EXTENSION IF NOT EXISTS pg_cron;           -- Schedule driver payout jobs

-- Example: ST_GeoHash for spatial bucketing (heatmaps)
SELECT ST_GeoHash(location, 5) AS geohash, COUNT(*) AS driver_count
FROM rides.driver_locations
WHERE is_available = TRUE
GROUP BY geohash;
```

13. Testing Guide

```
-- TEST 1: Find nearby drivers
SELECT * FROM rides.match_trip(
    1,
    ST_SetSRID(ST_MakePoint(-73.9857, 40.7580), 4326),
    1,
    5.0
);

-- TEST 2: Distance calculation
```

```

SELECT ST_Distance(
  ST_SetSRID(ST_MakePoint(-73.9857, 40.7580), 4326)::GEOGRAPHY,
  ST_SetSRID(ST_MakePoint(-73.9442, 40.6782), 4326)::GEOGRAPHY
) / 1609.34 AS distance_miles; -- Should be ~8.5 miles (JFK area)

-- TEST 3: Surge pricing
-- Run Q3 to see current surge. Then insert fake ride requests and rerun

-- TEST 4: Complete a trip
-- First update trip status to InProgress, then call complete_trip

-- TEST 5: Zone containment check
SELECT ST_Within(
  ST_SetSRID(ST_MakePoint(-73.9857, 40.7580), 4326),
  boundary
) AS in_zone
FROM rides.service_zones WHERE name = 'NYC Metro';
-- Should return TRUE

```

14. Extension Challenges

- Pool Matching:** Implement Pool/carpooling trips where up to 4 riders share a vehicle traveling similar routes. Write a function that finds eligible ongoing trips for a new pool request using `ST_DWithin` on both pickup and dropoff proximity.
- Driver Earnings Tax Report:** Build a quarterly earnings report for tax purposes per driver. Include gross earnings, platform fees, tips, mileage deduction estimate, and net taxable income. Support export in tabular format.
- Predictive ETA:** Build a `historical_travel_times` table that stores average travel time between geo-hash cells by hour of day and day of week. Write a query that uses this data to predict ETA more accurately than straight-line distance.
- Safety Corridor Alerts:** Add a `safe_zones` table with known safe/unsafe polygon areas. Write a trigger that fires when a trip route enters a flagged unsafe zone and inserts a safety alert.
- Driver Score Card:** Create a comprehensive weekly driver scorecard covering: rating trend, acceptance rate, cancellation rate, on-time pickup rate, income per hour. Use `NTILE` to rank drivers and send tier-change notifications.

15. What You Learned

Skill	Demonstrated By
PostGIS geospatial data	<code>GEOMETRY(POINT)</code> , <code>ST_DWithin</code> , <code>ST_Distance</code>
KNN operator <code><-></code>	Driver matching sorted by geography, not calculated distance
GIST spatial indexes	Sub-millisecond proximity searches
Dynamic pricing	Surge formula from supply/demand ratio

LISTEN/NOTIFY	Real-time driver location change broadcast
Fare calculation	Multi-component pricing with surge multiplier
Geofencing	ST_Within for zone membership checks
Real-time location table	driver_locations with upsert pattern
Driver analytics	Utilization, earnings, rating aggregations
NTILE ranking	Driver performance tier classification